CPSY 1291 · Recitation handout 10

Sequences, recurrence, and dynamics

Unrolling, vanishing gradients, fixed points, and a Hopfield network in fifteen lines

Fall 2026 · Week 10 · hold this after Lecture 15; Assignment 4 is due Tue 11/24

What this session is for

Two lectures, one object. Lecture 15 treats a recurrent network as a machine that processes sequences; Lecture 16 treats the same equations as a *dynamical system* with fixed points and basins of attraction. They are the same object viewed from two directions, and the second view is what lets you reverse-engineer a trained network instead of merely evaluating it.

This session is the code for both, plus the two things that reliably go wrong: shapes, and gradients that vanish.

> **The one idea.** A recurrent network is a *map* $\boldsymbol{h} \mapsto f(\boldsymbol{h}, \boldsymbol{x})$ applied over and over. Everything else follows from that: training difficulties come from differentiating a long composition of that map, and memory comes from the map having states it does not leave.

1 Shapes, before anything else

Object	Shape	Note
a batch of sequences	(N, T, D)	with `batch_first=True`
the same, PyTorch default	(T, N, D)	the default is *not* batch-first
hidden state	(layers, N, H)	note: batch is in the middle, always

> **Always pass `batch_first=True`** to `nn.RNN`, `nn.LSTM`, and `nn.GRU`, and note that the hidden state ignores it — h is (layers, N, H) either way. That inconsistency is the single most common source of silent shape bugs in recurrent code. It is silent because (T, N, D) and (N, T, D) have the same number of elements, so nothing errors when T and N happen to be compatible; you simply train on transposed data.

2 A recurrent cell, by hand

$$\boldsymbol{h}_t = \tanh(\boldsymbol{W}_{hh}\boldsymbol{h}_{t-1} + \boldsymbol{W}_{xh}\boldsymbol{x}_t + \boldsymbol{b})$$

```
class RNNCell(nn.Module):
   def __init__(self, d_in, d_hid):
      super().__init__()
```

```
        self.Wxh = nn.Linear(d_in, d_hid)
        self.Whh = nn.Linear(d_hid, d_hid, bias=False)

    def forward(self, x, h):
        return torch.tanh(self.Wxh(x) + self.Whh(h))

h = torch.zeros(N, d_hid)
for t in range(T):
    h = cell(X[:, t], h) # the SAME weights at every step
```

Two things to notice in that loop. The weights do not change with t — that is **weight sharing across time**, the temporal analogue of a convolution's weight sharing across space, and it is what lets one network handle sequences of any length. And the loop builds a graph T layers deep: **unrolling**. An RNN run for 50 steps is a 50-layer feedforward network whose layers happen to be identical.

2.1 The library version

```
rnn = nn.LSTM(input_size=D, hidden_size=H, num_layers=1, batch_first=True)
out, (h_n, c_n) = rnn(X) # out: (N, T, H) h_n: (1, N, H)
```

`out` holds the hidden state at *every* step; `h_n` holds only the last one. Which you want depends on the task: classifying a whole sequence uses `h_n` (or a pooled `out`); labeling every step uses `out`.

3 Why gradients vanish, and how to see it

Differentiating a loss at step T with respect to a weight at step 1 requires the chain rule through every intervening step, which produces a *product* of T Jacobians. Products of many numbers below 1 go to zero; products of numbers above 1 explode. That is the whole phenomenon.

```
loss.backward()
for t, h in enumerate(hidden_states):
    print(t, h.grad.norm().item()) # requires h.retain_grad() beforehand
```

Plot those norms against t on a log axis. A clean exponential decay backwards in time is the vanishing gradient made visible, and it is worth seeing once — it converts a piece of received wisdom into an observation.

Problem	What you do about it
exploding	`clip_grad_norm_(model.parameters(), 1.0)` — reliable, one line
vanishing	gating: LSTM or GRU. Clipping does not help

Gating works because the cell state passes through an *addition* rather than a repeated multiplication, so the gradient has a path back through time that is not a long product. That is the mechanism; the three gates are the bookkeeping around it.

3.1 Teacher forcing

When training a model to generate a sequence, you can feed back either the model's own output or the true previous token. Feeding the truth is **teacher forcing**: it trains far faster and more stably, and it creates a mismatch — at generation time the model sees its own outputs, including its own mistakes, which it never saw during training. Errors then compound. This is worth knowing before Lecture 17, where the same trick trains every language model.

4 The same equations as a dynamical system

Set the input to a constant and the update becomes a map from $\boldsymbol{h}$ to $\boldsymbol{h}$. Iterate it and ask where it goes.

4.1 Finding fixed points numerically

A **fixed point** satisfies $\boldsymbol{h}^* = f(\boldsymbol{h}^*)$. Find them by minimizing $q(\boldsymbol{h}) = \frac{1}{2}\|f(\boldsymbol{h}) - \boldsymbol{h}\|^2$ from many random starting points:

```
h = torch.randn(200, H, requires_grad=True) # 200 starting points at once
opt = torch.optim.Adam([h], lr=0.01)
for _ in range(2000):
   opt.zero_grad()
   q = 0.5 * ((cell(x_const, h) - h) ** 2).sum(1)
   q.sum().backward()
   opt.step()
found = h[q.detach() < 1e-8] # keep the ones that converged
```

This is the standard method for reverse-engineering a trained RNN, and it is short because autograd does the work. Starting from many points matters: a network can have several fixed points, and each one has its own basin.

4.2 Stable or unstable?

Linearize at a fixed point — take the Jacobian $\boldsymbol{J} = \partial f / \partial \boldsymbol{h}$ evaluated there — and look at its eigenvalues.

All $\lvert\lambda\rvert < 1$	**stable**: nearby states fall in. A memory
Some $\lvert\lambda\rvert > 1$	**unstable** in those directions: a saddle
One $\lvert\lambda\rvert \approx 1$	a **line attractor**: the state drifts freely along it

The third row is the interesting one for neuroscience: a direction that neither decays nor grows is a way to hold a continuous quantity, which is how a network integrates evidence or holds a value in working memory.

```
J = torch.autograd.functional.jacobian(lambda h: cell(x_const, h), h_star)
np.abs(np.linalg.eigvals(J.numpy()))
```

5 A Hopfield network in fifteen lines

The clearest case of a network whose memories *are* its attractors.

```
def store(patterns): # patterns: (P, n) of +/-1
   W = patterns.T @ patterns / len(patterns) # Hebbian outer product
   np.fill_diagonal(W, 0) # no self-connections
   return W

def recall(W, s, steps=20, rng=np.random.default_rng(0)):
   s = s.copy()
   for _ in range(steps):
      for i in rng.permutation(len(s)): # asynchronous update
         s[i] = 1 if W[i] @ s >= 0 else -1
   return s

def energy(W, s):
   return -0.5 * s @ W @ s
```

Three points to check by running it. Recall **monotonically decreases the energy**, which is why it converges rather than cycling — and asynchronous updating is required for that guarantee. The **capacity** is about $0.14n$ patterns for n units; store more and recall returns blends. And the **negation** of every stored pattern is also a fixed point, which falls straight out of the symmetry of the update rule.

6 Exercises

1. You pass (N, T, D) data to `nn.LSTM` without `batch_first=True`. Nothing errors. What is the model learning?

2. Your RNN trains fine on sequences of length 10 and not at all on sequences of length 200. Name the mechanism, and the architectural fix.

3. Gradient clipping at 1.0 does not help a network whose loss is flat from epoch 1. Why not?

4. You find 40 candidate fixed points from 200 random starts, but 160 of them have $q > 10^{-3}$. What are those 160, and should you report them?

5. A trained RNN has a fixed point whose Jacobian has one eigenvalue at 0.999 and the rest below 0.6. What is the network doing?

6. A Hopfield network with 100 units is given 30 patterns to store. What happens on recall, and why?

Answers

1. It reads your batch dimension as time and your time dimension as batch: each "sequence" is one time step across N different examples. It trains, it produces a loss, and it is learning

nothing about temporal structure. Nothing errors because both layouts contain the same number of elements.

2. Vanishing gradients: the gradient from step 200 back to step 1 is a product of 200 Jacobians. Fix: gating — an LSTM or GRU — because the cell state passes through an addition rather than a repeated multiplication.

3. Clipping bounds gradients from above; it does nothing to gradients that are too small. A flat loss from the start is vanishing gradients, a dead nonlinearity, or a learning rate far too small — none of which clipping addresses.

4. They are starting points that did not converge — the optimizer was still descending, or it was heading toward a region with no fixed point. Discard them, and report both numbers: "40 fixed points found from 200 random initializations". Reporting only the 40 hides how thoroughly you searched.

5. It has a **line attractor**: one direction along which the state neither decays nor grows, and every other direction contracting onto it. That is how a network holds a continuous quantity — integrating evidence, or maintaining a value in working memory.

6. Capacity is about $0.14 \times 100 = 14$ patterns, so 30 is roughly double it. Recall converges to spurious states — blends and mixtures of stored patterns — rather than to the patterns themselves, and the failure gets worse the more you store.

Cheat sheet

You want	You write
sequences shaped how you think	`nn.LSTM(..., batch_first=True)`
every step's hidden state	`out, _ = rnn(X)` → `(N, T, H)`
only the last	`_, (h_n, _) = rnn(X)` → `(1, N, H)`
to stop exploding gradients	`clip_grad_norm_(m.parameters(), 1.0)`
to stop vanishing gradients	use gating; clipping will not help
gradient norms through time	`h.retain_grad()`, then `h.grad.norm()`
fixed points	minimize $\frac{1}{2}\|f(h) - h\|^2$ from many starts
stability at a fixed point	eigenvalues of the Jacobian; $\|\lambda\| < 1$ is stable
a Hopfield weight matrix	`P.T @ P / len(P)`, zero the diagonal
Hopfield capacity	about $0.14n$ patterns